

Problem Set — Week 4

Computability · VCE Algorithmics (HESS) · Unit 4 AOS3

Sections A–C correspond to Topics 12–14. Attempt each after its lesson.

Name: _____

Due: Tuesday 11 August

Precision warning: this topic is marked on the exactness of your wording. Undecidability is a claim about the non-existence of *one algorithm covering every instance* — not about individual instances being unsolvable. Begin such answers with “there is no **single** algorithm that...”

Section A — Turing machines

(Topic 12)

A1. Name the four components of a Turing machine and state, for each, what it corresponds to in a modern computer.

A2. Consider the machine M with the transition table below. Entries are written *write, move, next state*; the blank symbol is $_$ and H is the halt state.

State	reads $_$	reads 0	reads 1
A	$_,$ left, B	1, right, A	0, right, A
B	$_,$ right, H	0, left, B	1, left, B

The machine starts in state A with the head on the **leftmost** cell of the tape 101.

- Trace the machine. Give your working as a table with columns *step, state, symbol read, action, tape after*.
- State the final tape contents and where the head finishes.
- In one sentence, describe what M computes. Why does it have two states rather than one?

A3. Write the transition table for a Turing machine that **erases its tape** — replacing every symbol with a blank — and then halts. State your alphabet and where the head starts.

A4. A student claims: “*Turing machines can only run very simple algorithms.*” Explain why this is wrong, distinguishing the machine’s *architecture* from its *capability*.

Section B — The Halting Problem and undecidability

(Topic 13)

B1. Define *decidable* and *undecidable*. Your definitions must make clear both requirements on the algorithm.

B2. State the Halting Problem precisely: what is the input, what is the question, and what did Turing prove?

B3. Outline Turing's proof in five moves. Label them: assume, construct, ask, case analysis, conclude.

B4. A student explains: *"An undecidable problem is one where there is no instance of the problem that can be solved."* Explain why this is incorrect and give an improved explanation. Include a specific instance of the Halting Problem that can be settled.

B5. Classify each statement as true or false, with a one-line justification.

- a.) Undecidable problems are decision problems.
- b.) There exist algorithms that solve many instances of an undecidable problem.
- c.) An intractable problem is also undecidable.
- d.) A decidable problem always has a polynomial-time algorithm.

B6. Explain why a debugger cannot warn you about every infinite loop in every program, and describe what real static-analysis tools do instead.

Section C — The Church–Turing thesis

(Topic 14)

C1. State the Church–Turing thesis, and explain why it is called a *thesis* rather than a theorem.

C2. A technology journalist writes that quantum computers will eventually “solve problems like the Halting Problem that today’s machines cannot”. Write a two-paragraph correction, distinguishing computability from complexity.

C3. Three separate questions can be asked about any problem: is it *computable*, is it *feasible*, is it *practical*? Define each, and explain why they must be answered in that order.

C4. Exam-style. Two students are discussing what they have learned.

Student 1: “Turing showed Hilbert’s programme was impossible by proving there are some things Turing machines cannot do.”

Student 2: “Yes — Turing showed that Turing machines can only run algorithms that terminate in polynomial time.”

Explain why Student 2 is incorrect. Use the connection between Hilbert’s programme and Turing’s work to support your answer.

C5. Explain why the Church–Turing thesis is what makes undecidability results *permanent*, rather than statements about the technology of the 1930s.

Section D — Consolidation

D1. Place each problem in the correct region of the diagram: *tractable*, *decidable but intractable*, or *undecidable*. Justify the two you are least sure of.

- | | |
|---|--|
| a.) Finding a minimum spanning tree | d.) Sorting a list |
| b.) The travelling salesman problem | e.) Graph colouring |
| c.) Determining whether a program halts | f.) Deciding whether two programs are equivalent |
